Process Synchronization

Objectives and Outline

Objectives

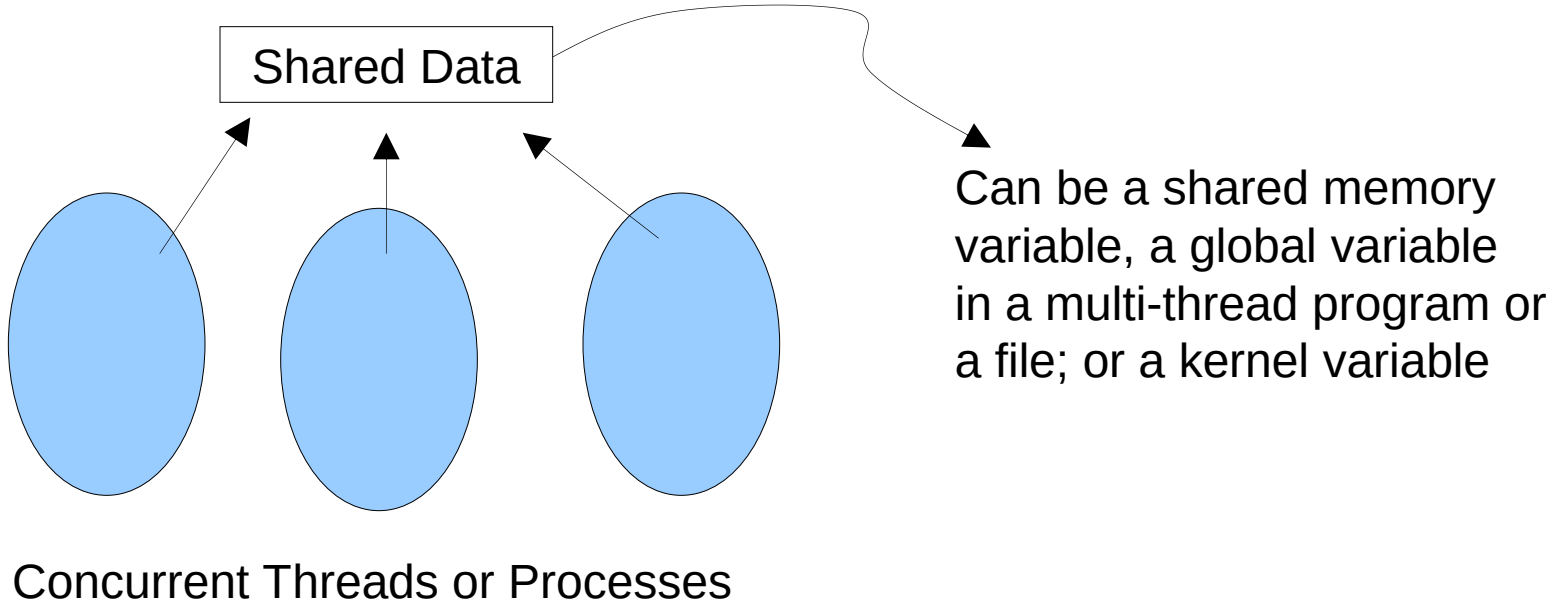
- To introduce the **critical-section** problem
- Critical section solutions can be used to ensure the **consistency** of shared data
- To present both **software and hardware solutions** of the critical-section problem

Outline

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization
- Monitors

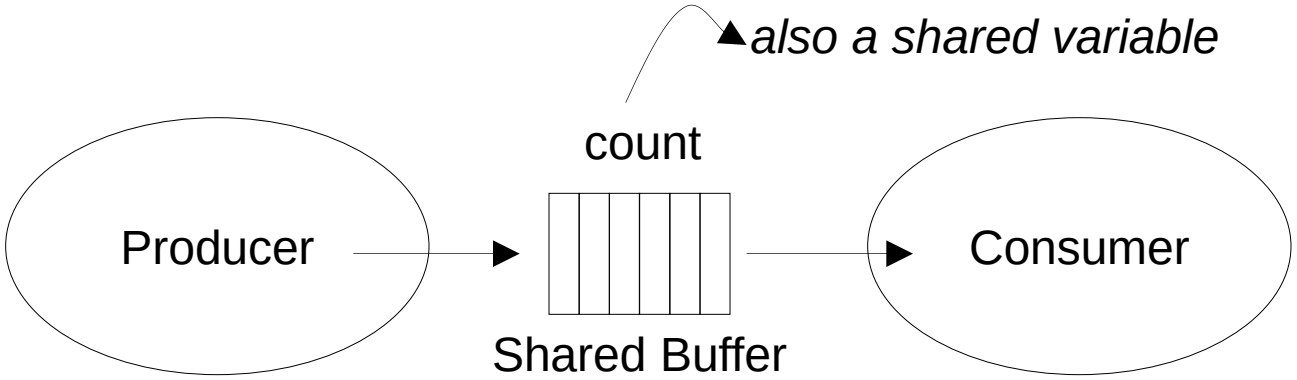
Background

- **Concurrent** access to **shared data** may result in **data inconsistency**
- Maintaining data consistency requires mechanisms to ensure the **orderly execution** of cooperating processes



Producer Consumer Problem

- Below is a solution to the consumer-producer problem that fills **all** the slots of the **shared buffer**.
- Use an integer **count** to keep track of the number of **full slots**.
- Initially, count is set to 0. It is **incremented** by the producer after it puts a new item and is **decremented** by the consumer after it retrieves an item from the buffer



at most BUFFER_SIZE items

Producer and Consumer Code

PRODUCER

```
while (true) {
        /*produce an item*/
         nextProduced  = ….

         while (count == BUFFER_SIZE)
                 ; // do nothing

         buffer [in] = nextProduced;
         in = (in + 1) % BUFFER_SIZE;
         count++;
}
```

CONSUMER

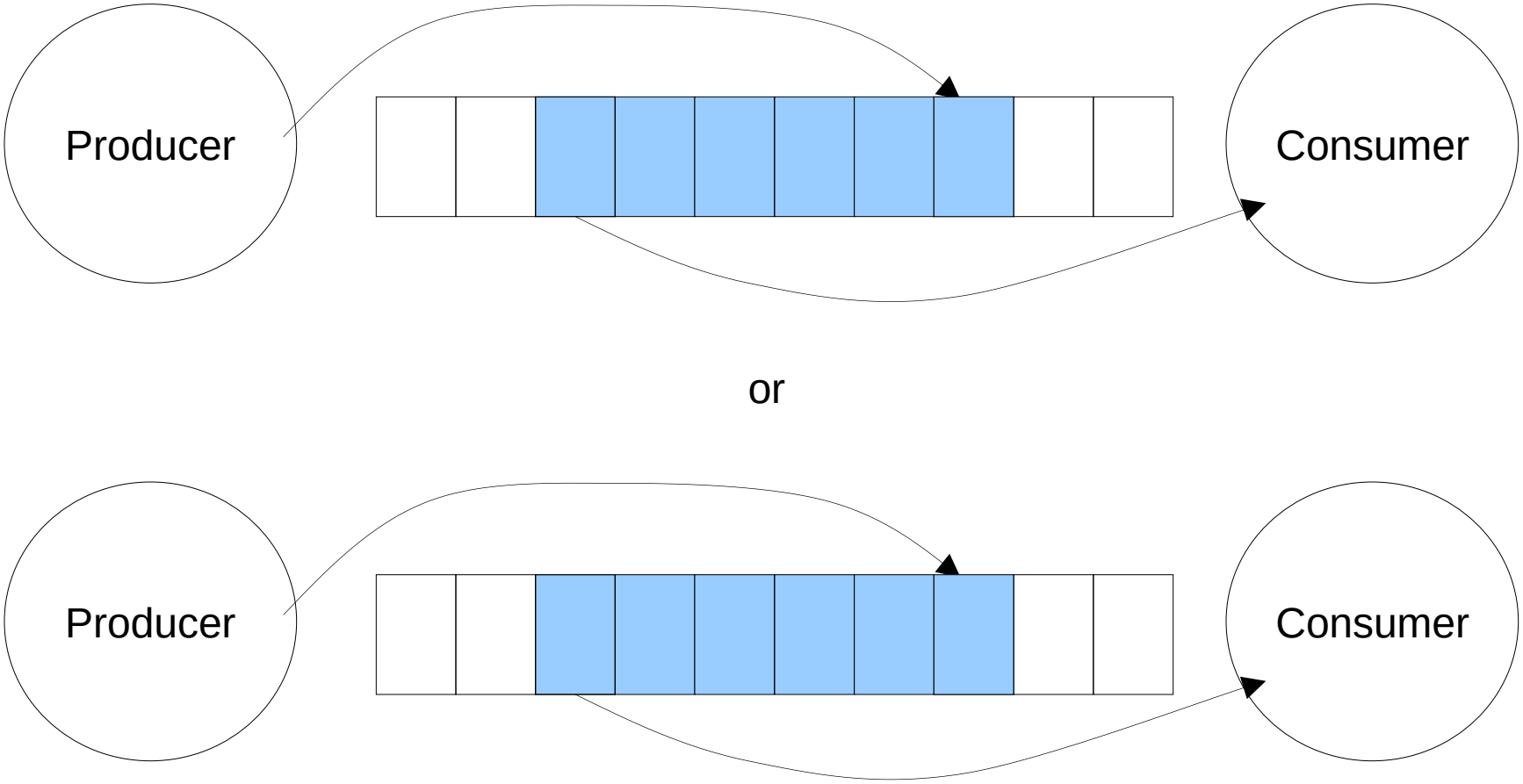
```
while (true)  {
       while (count == 0)
               ; // do nothing

       nextConsumed =  buffer[out];
       out = (out + 1) % BUFFER_SIZE;
       count--;

        /*consume item nextConsumed*/
}
```

a possible Problem: race condition

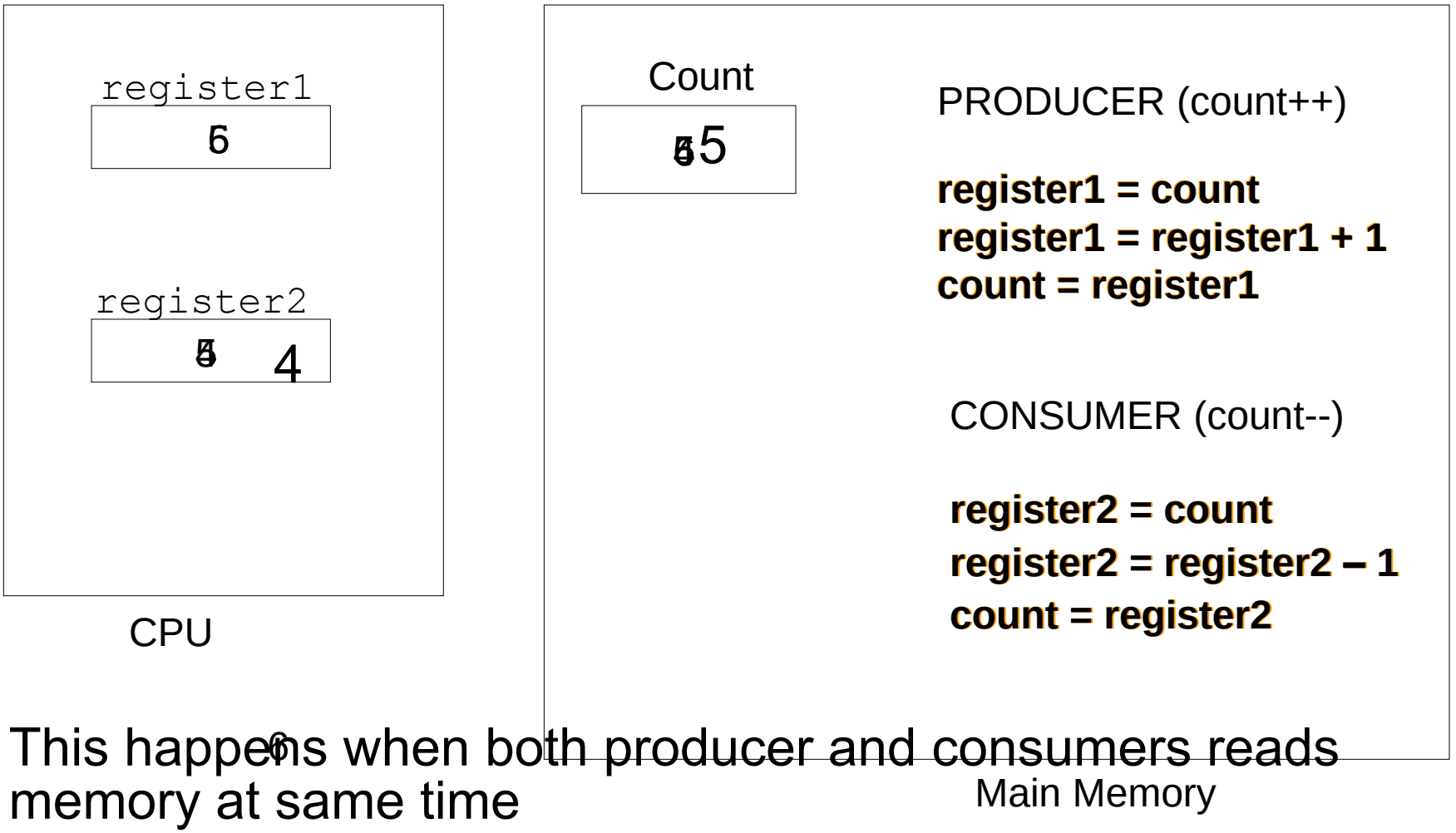
- Assume we had 5 items in the buffer
- Then:
  - Assume producer has just produced a new item and put it into buffer and is about to increment the count.
  - Assume the consumer has just retrieved an item from buffer and is about the decrement the count.
  - Namely: Assume producer and consumer is now about to execute count++ and count– statements.


Producer
Consumer
or
Producer
Consumer

Race Condition

- **count++** could be implemented as
  register1 = count
  register1 = register1 + 1
  count = register1
- **count--** could be implemented as
  register2 = count
  register2 = register2 - 1
  count = register2

Race Condition



This happens when both producer and consumers reads memory at same time

Interleaved Execution sequence

- Consider this execution interleaving with “count = 5” initially:

  S0: producer execute register1 = count {register1 = 5}
  S1: producer execute register1 = register1 + 1 {register1 = 6}
  S2: consumer execute register2 = count {register2 = 5}
  S3: consumer execute register2 = register2 - 1 {register2 = 4}
  S4: producer execute count = register1 {count = 6 }
  S5: consumer execute count = register2 {count = 4}

Race Condition

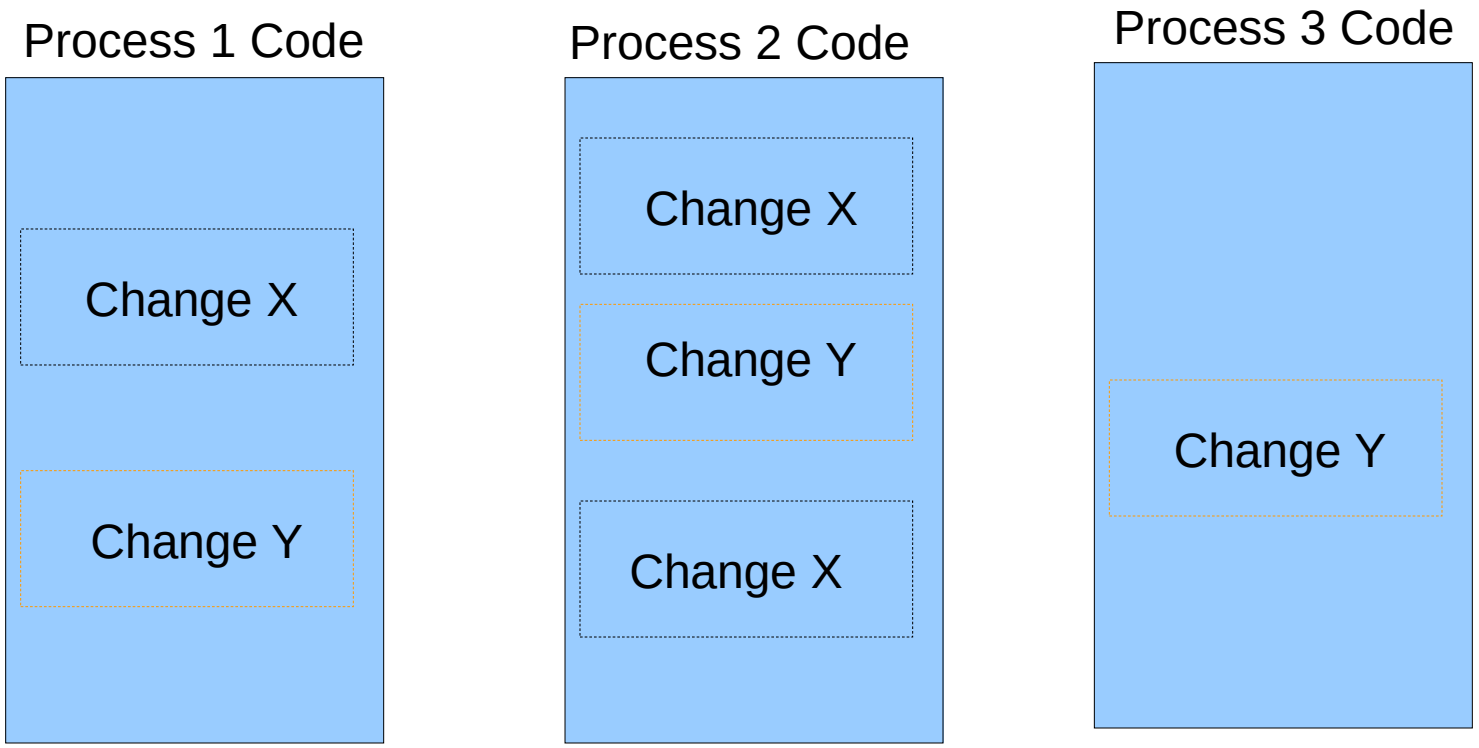
- Race condition occurs when output depends on sequence or timing of uncontrolled events.

- i.e. a race condition arises if multiple threads of execution enter the critical section at roughly the same time; both attempt to update the shared data structure, leading to a surprising (and perhaps undesirable) outcome.

- We may get a different result each time! This result is called indeterminate.

Programs and critical sections

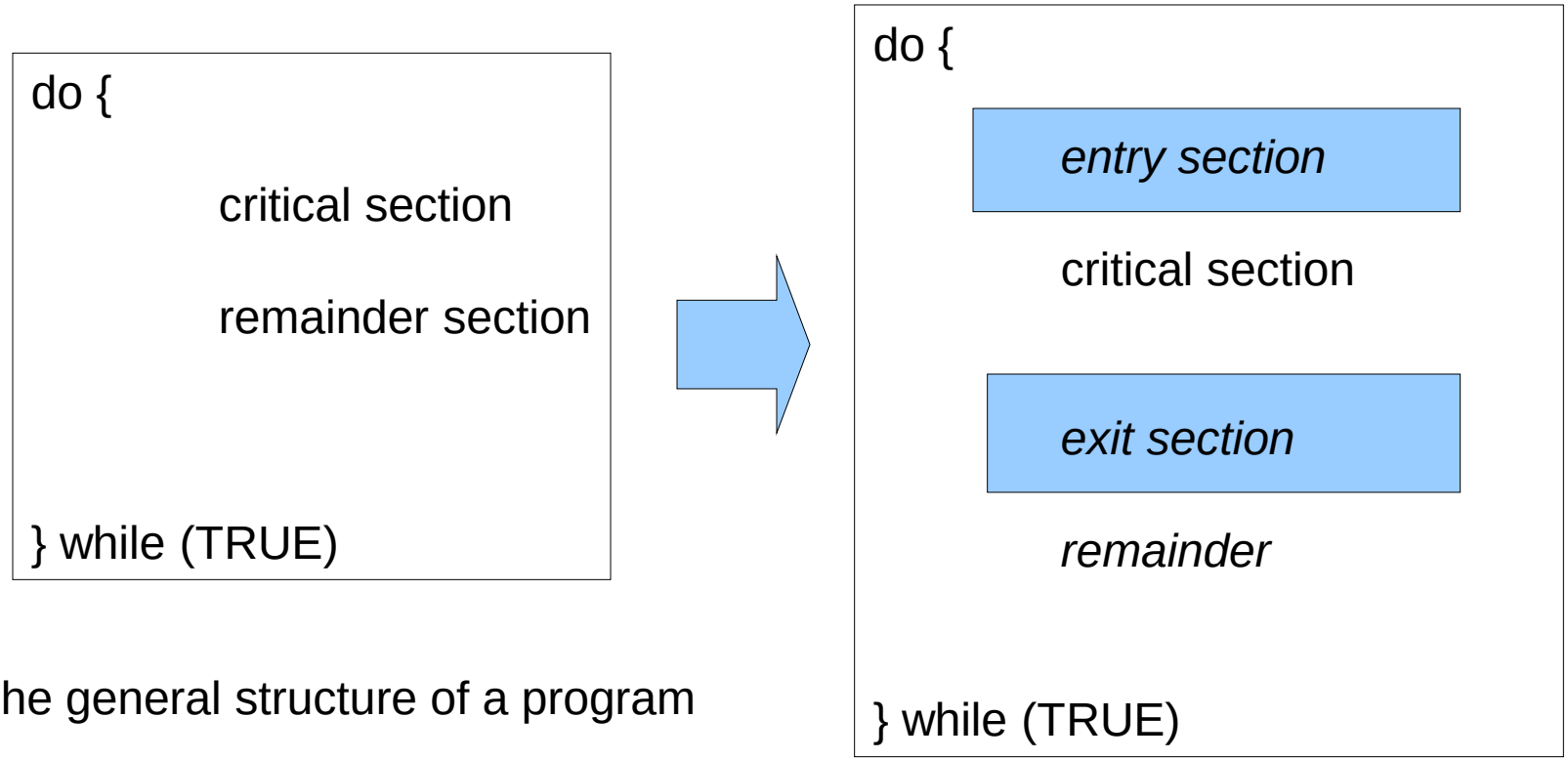
- The part of the program (process) that is accessing and changing shared data is called its critical section



Assuming X and Y are shared data.

Structuring Programs

- The general way to do that is:



The general structure of a program

Entry section will allow only one process to enter and execute critical section code.

Critical-Section Properties

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections

2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely ***//* no deadlock**

3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted ***//* no starvation of a process**

Applications and Kernel

- Multiprocess applications sharing a file or shared memory segment may face critical section problems.
- Multithreaded applications sharing global variables may also face critical section problems.
- Similarly, kernel itself may face critical section problem. It may have critical sections.

Peterson's Solution

- Two process solution
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted.
- The two processes share two variables:
  - int **turn;**
  - Boolean **flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section.
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i]** = true implies that process $\mathbf{P_i}$ is ready!

Algorithm for Process P_i

```
do {
        flag[i] = TRUE;
        turn = j;
        while (flag[j] && turn == j);
                critical section
        flag[i] = FALSE;
                remainder section

} while (1)
```

entry section

exit section

Two processes executing concurrently

PROCESS i (0)

```
do {
        flag[i] = TRUE;
        turn = j;
        while (flag[j] && turn == j);
        critical section…..
        flag[i] = FALSE;
        remainder section…..
}  while (1)
```

PROCESS j (1)

```
do {
        flag[j] = TRUE;
        turn = i;
        while (flag[i] && turn == i);
        critical section…..
        flag[j] = FALSE;
        remainder section…..
} while (1)
```

Shared Variables } **flag[ ]**, **turn**

Synchronization Hardware

- We can use some **hardware support** (if available) for protecting critical section code
  - 1) Disable interrupts? maybe
    - Sometimes only (kernel)
    - Not on multiprocessors
  - 2) Special machines instructions and **lock variables**
    - TestandSet
    - Swap

Synchronization Hardware

- **Use of lock variables** is a general and very common approach.

- What happens if you a lock variable without special instructions?
  - Can be source of race conditions
    - Example:

      int lock = 0; // global variable (shared among threads)

Thread 1

```
while (lock == 1)
            ; // loop
lock =  1;
// critical section
lock = 0;
```

Thread 2

```
while (lock == 1)
            ; // loop
lock =  1;
// critical section
lock = 0;
```

above code is NOT correct solution

Solution to Critical-section Problem Using Locks

```
do {
    acquire lock
    critical section
    release lock
    remainder section
} while (TRUE);
```

Only one process can acquire lock. Others has to wait (or busy loop)

Synchronization Hardware

- Therefore we need to use special machine instructions that can do testing and setting atomically or something like that (like swapping)

- Some possible atomic (non-interruptable) machine instructions:
  - **TestAndSet** instruction (TSL):
    test memory word and set value

  - **Swap** instruction (EXCH instruction in Intel x86 arch):
    swap contents of two memory words

- They can be executed **atomically** in a **multi-processor environment** as well

TestAndSet Instruction

- Is a machine/assembly instruction.
  - But here we provide definition of it using a high level language code.

Definition of TestAndSet Instruction

```
boolean TestAndSet (boolean *target)
 {
        boolean rv = *target;
        *target = TRUE;
        return rv:
}
```

Solution using TestAndSet

- To use it, we need to program in assembly language.
  i.e., entry section code should be programmed in assembly

Solution

We use a shared Boolean variable **lock**, initialized to **false**.

```
do {
        while ( TestAndSet (&lock ))
                ;   // do nothing

        //    critical section

        lock = FALSE;

        //      remainder section

} while (TRUE);
```

entry section

exit_section

```
boolean TestAndSet (boolean *target)
{       boolean rv = *target;
        *target = TRUE;
        return rv:   }
```

Swap Instruction

- Is a machine/assembly instruction. Intel 80x86 architecture has an XCHG instruction
  - But here we provide definition of it using a high level language code.

Definition of Swap Instruction

```
void Swap (boolean *a, boolean *b)
 {
            boolean temp = *a;
             *a = *b;
             *b = temp:
}
```

Solution using Swap

- Need to program in assembly to use. Hence Entry section code should be programmed in assembly

Solution

We use a shared Boolean variable **lock** initialized to FALSE.

Each process also has a local Boolean variable **key**

```
do {
        key = TRUE;
        while ( key == TRUE)
                Swap (&lock, &key );

        //    critical section

        lock = FALSE;

        //      remainder section

} while (TRUE);
```

Comments

- TestAndSet and Swap provides mutual exclusion: 1st property satisfied
- But, Bounded Waiting property, 3rd property, may not be satisfied.
- A process X may be waiting, but we can have the other process Y going into the critical region repeatedly

Bounded-waiting Mutual Exclusion with TestandSet()

```
do {
        waiting[i] = TRUE;
        key = TRUE;
        while (waiting[i] && key)
                key = TestAndSet(&lock);
        waiting[i] = FALSE;

        // critical section

        j = (i + 1) % n;
        while ((j != i) && !waiting[j])
                j = (j + 1) % n;
        if (j == i)
                lock = FALSE;
        else
                waiting[j] = FALSE;
        // remainder section
} while (TRUE);
```

entry section code

exit section code

Semaphore

- Synchronization tool that does not require busy waiting
- Semaphore *S*: integer variable
  shared, and can be a kernel variable
- Two standard operations modify **S:** **wait()** and **signal()**
  - Originally called **P()** and **V()**
  - Also called **down() and up()**
  - Semaphores can only be accessed via these two indivisible (atomic) operations;
  - They can be implemented as system calls by kernel. Kernel makes sure they are **indivisible**.

- Less complicated entry and exit sections when semaphores are used

Meaning (semantics) of operations

- **wait (S):**

  if S positive

  S-- and return

  else

  block/wait (*until somebody wakes you up; then return*)

- **signal(S):**

  if there is a process waiting

  wake it up and return

  else

  S++ and return

Comments

- Wait body and signal body have to be executed atomically: one process at a time. Hence the body of wait and signal are critical sections to be protected by the kernel.

- Not that when wait() causes the process to block, the operation is nearly finished (wait body critical section is done).

- That means another process can execute wait body or signal body

Semaphore as General Synchronization Tool

- **Binary** semaphore – integer value can range only between 0 and 1; can be simpler to implement
  - Also known as **mutex locks**
  - Binary semaphores provides mutual exclusion; can be used for the critical section problem.
- **Counting** semaphore – integer value can range over an unrestricted domain
  - Can be used for other synchronization problems; for example for resource allocation.

Usage

- Binary semaphores (mutexes) can be used to solve critical section problems.
- A semaphore variable (lets say **mutex**) can be shared by N processes, and **initialized to 1.**
- Each process is structured as follows:

```
do {
        wait (mutex);
         // Critical Section
        signal (mutex);
        // remainder section
} while (TRUE);
```

usage: mutual exclusion

Process 0

```
do {
        wait (mutex);
          // Critical Section
        signal (mutex);
        // remainder section
} while (TRUE);
```

Process 1

```
do {
        wait (mutex);
        // Critical Section
        signal (mutex);
        // remainder section
} while (TRUE);
```

Kernel

```
wait() {…}          signal() {…}

Semaphore mutex;    //  initialized to 1
```

usage: other synchronization problems

P0

```
…
S1;
….
```

P1

```
…
S2;
….
```

Assume we definitely want to have S1 executed before S2.

semaphore x = 0; // initialized to 0

Solution:

P0

```
…
S1;
signal (x);
….
```

P1

```
…
wait (x);
S2;
….
```

usage: resource allocation

- Assume we have a resource that has 5 instances. A process that needs that type of resource will need to use one instance. We can allow at most 5 process concurrently using these 5 resource instances. Another process (processes) that want the resource need to block. How can we code those processes?
- Solution:

one of the processes creates and initializes a semaphore to 5.

semaphore x = 5; // semaphore to access resource

```
wait (x);
…
….use one instance
of the resource…
…
signal (x);
```

Each process has to be coded in this manner.

Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let S and Q be two semaphores initialized to 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

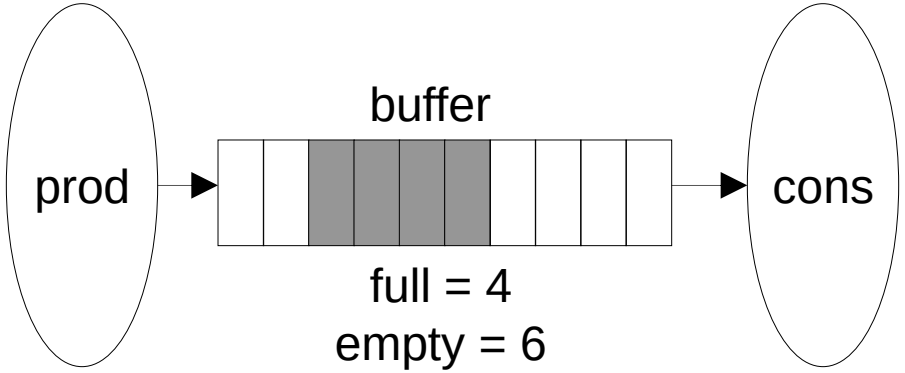
- **Starvation** – indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** - Scheduling problem when lower-priority process holds a lock needed by higher-priority process

Classical Problems of Synchronization

- Bounded-Buffer Problem
- Dining-Philosophers Problem

Bounded Buffer Problem

- *N* buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value N.



Bounded Buffer Problem

The structure of the **producer** process

```
do  {
            //   produce an item in nextp

            wait (empty);
            wait (mutex);

            //  add the item to the  buffer

            signal (mutex);
            signal (full);

} while (TRUE);
```

The structure of the **consumer** process

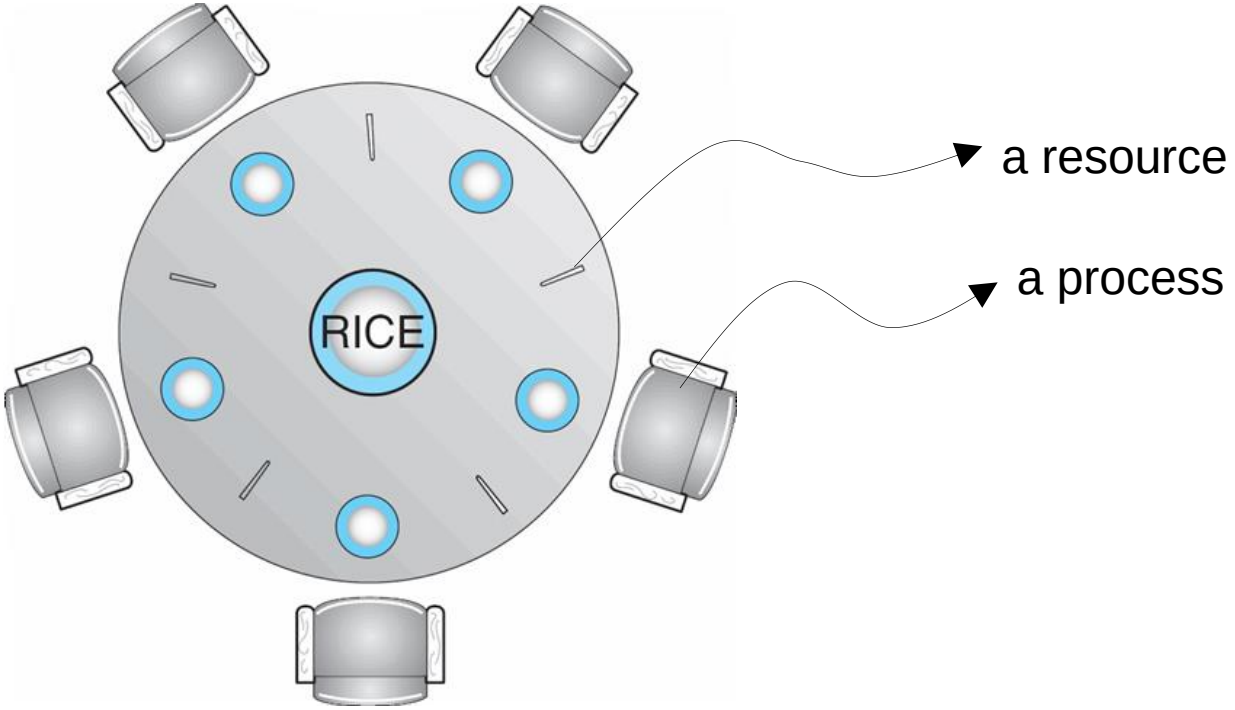
```
do {
            wait (full);
            wait (mutex);

            //  remove an item from
            //  buffer to nextc

            signal (mutex);
            signal (empty);

            //  consume the item in nextc
} while (TRUE);
```

Dining-Philosophers Problem



Assume a philosopher needs two forks to eat. Forks are like resources.
While a philosopher is holding a fork, another one can not have it.

Dining-Philosophers Problem

- Is not a real problem
- But lots of real resource allocation problems look like this. If we can solve this problem effectively and efficiently, we can also solve the real problems.
- From a satisfactory solution:
  - We want to have concurrency: two philosophers that are not sitting next to each other on the table should be able to eat concurrently.
  - We don’t want deadlock: waiting for each other indefinitely.
  - We don’t want starvation: no philosopher waits forever.

Dining-Philosophers Problem (Cont.)

Semaphore **chopstick [5]** initialized to 1

```
do  {
        wait ( chopstick[i] );
        wait ( chopStick[ (i + 1) % 5] );

        //  eat

        signal ( chopstick[i] );
        signal (chopstick[ (i + 1) % 5] );

        //  think
} while (TRUE);
```

This solution provides concurrency but may result in deadlock.

Monitors

- The monitor is a programming-language construct that provides equivalent functionality to that of semaphores and that is easier to control.
- Implemented in a number of programming languages, including
  - Concurrent Pascal, Pascal-Plus,
  - Modula-2, Modula-3, and Java.

Chief characteristics

- Local data variables are accessible only by the monitor
- Process enters monitor by invoking one of its procedures
- Only one process may be executing in the monitor at a time

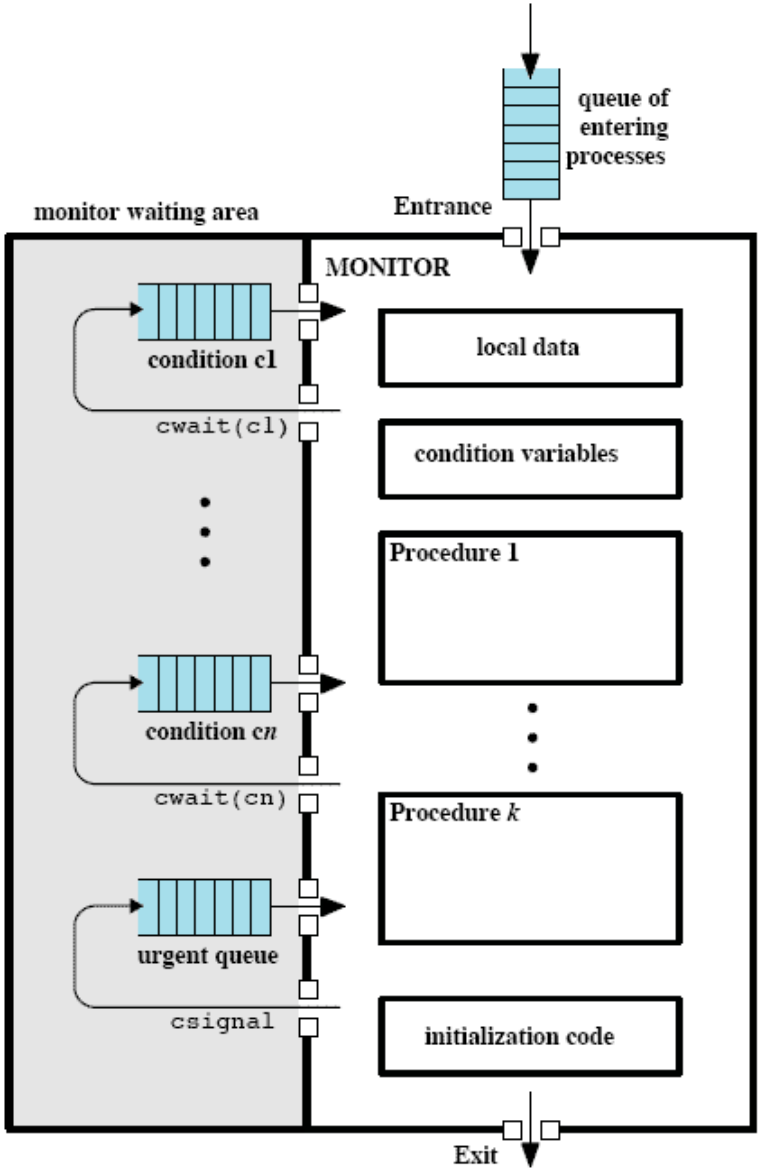
Synchronization

- Synchronisation achieved by **condition variables** within a monitor
  - only accessible by the monitor.

- **condition x, y;**

- Two operations on a condition variable:
  - **x.wait ()** – a process that invokes the operation is suspended.
  - **x.signal ()** – resumes one of processes (if any) that invoked **x.wait ()**

Structure of a Monitor



Condition variables: example

- Lets us do an example.

- Assume we have a resource to be accessed by many processes. Assume we have 5 instanced of the resource. 5 processes can use the resource simultaneously.

- We want to implement a monitor that will implement two functions: acquire() and release() that can be called by a process before and after using a resource.

Condition variables: example

```
monitor Allocate
{
        int count = 5;  // we initialize count to 5.
        condition c;

        void acquire () {
                if (count == 0)
                        c.wait();
                count--;
        }

        void release () {
                count++;
                c.signal();
        }
}
```

Condition variables: example

A process will be coded like the following:

Allocate MA; // resource allocation monitor

….
MA.acquire();

// ….use the resource …

MA.release();

….